PYTHON PARA PROFESIONALES (INTERMEDIO) (ED. 2025-2026)

Aulas > Object-Oriented Programming

Examen: Test Object-Oriented Programming

La puntuación máxima para este cuestionario es de 10 puntos.

Cada pregunta que sea errónea resta **1/4** del valor de la pregunta.

El tiempo de realización de este examen está limitado a **50 minutos**
Una vez transcurrido este tiempo, el examen se entregará automáticamente con las preguntas que hayan sido contestadas hasta ese momento.

Tiempo restante para hacer el cuestionario: **00:24:28**

És imprescindible connectar-se al Meet com en les classes lectives i fer el test amb la cambra encesa.

Es imprescindible conectarse al Meet como en las clases lectivas y hacer el test con la cámara encendida.

Enviar

1 - In the following UML association:

Student "1" --> "0..*" Course

what does the arrow indicate?

- ✓ Student objects know about Course objects
- ○ Course objects know about Student objects
- ○ The relationship is composition
- ○ Both classes know about each other

2 - Structured programming focuses mainly on:

- ✓ organizing control flow using sequence, selection, and iteration
- ○ organizing the code using the divide and conquer principle
- ○ organizing data using nested structures
- ○ organizing data inside classes to model real-world entities

3 - What is a class attribute?

- ○ A variable defined in the class that is overridden by instance attributes
- ○ A variable that can only be accessed through instances, not the class itself
- ✓ A variable shared by all instances of a class
- ○ A variable defined only inside __init__

4 - Which of the following statements is true?

- ○ A static method can only be called from instances
- ○ A class method is used only to define utility functions that do not depend on any data
- ✓ A static method is independent of class and instance data
- ○ A class method cannot be inherited by subclasses

5 - Why are class methods useful?

- ✓ They allow operations on shared class state or alternative constructors
- ○ They provide a mechanism to override instance methods
- ○ They ensure that all subclasses share identical implementations of methods
- ○ They enable direct access to instance-specific data without creating objects

6 - What is polymorphism?

- ○ The mechanism by which a class inherits methods from multiple parent classes
- ○ The process of defining multiple methods with the same name but different implementations in the same class
- ○ The use of a common base class to enforce identical behavior across all subclasses
- ✓ Different objects responding to the same method call in different ways

7 - Given the following code, what is the result of c.start()?

```
class Vehicle:
    def __init__(self):
        self.type = "Vehicle"
    def start(self):
        return f"{self.type} starting"
class Car(Vehicle):
```

```
    def __init__(self):
        super().__init__()
        self.type = "Car"
    def start(self):
        return f"{self.type} starting"
c = Car()
```

- ○ Vehicle starting Car
- ○ None of the others
- ✓ Car starting
- ○ Vehicle starting

8 - Which statement best describes an interface in Python?

- ✓ An abstract class with only abstract methods
- ○ A class with only static methods
- ○ A base class that enforces inheritance but allows partial implementation by default
- ○ A class without attributes

9 - A single leading underscore in an attribute name indicates:

- ◯ attribute can only be modified under certain circumstances
- ✓ intended internal use
- ◯ attribute can only be initialized and should never be modified
- ◯ constant value

10 - Given the following code, what happens?

```
class A:
    def __init__(self):
        self.x = 10
class B(A):
    def __init__(self):
        self.y = 20
b = B()
```

- ◯ SyntaxError
- ◯ b.y is not created
- ✓ AttributeError when accessing b.x
- ◯ b.x is automatically set to 10

11 - What is the role of super() in a child class?

- ✓ To call methods from the parent class
- ○ To replace inherited attributes
- ○ To override methods
- ○ To create a new instance of the parent class

12 - Consider the following UML relationship from an online store example:

Order "1" *--> "1..*" OrderLine

Which interpretation is correct?

- ○ OrderLine objects exist independently and are not tied to an Order
- ✓ An Order is composed of one or more OrderLine objects
- ○ An Order may reference at most one OrderLine
- ○ An OrderLine inherits from Order

13 - Which of the following is a common reason to use UML diagrams before implementation?

- ○ To avoid writing documentation
- ○ To make the program run faster
- ✓ To understand and communicate system design

- ◯ To guarantee that the final code has no bugs

14 - If a class does not define an __init__ method:

- ◯ a syntax error occurs
- ✓ Python supplies a default constructor that takes no arguments and does nothing
- ◯ the instance will not have attributes
- ◯ creating an instance raises a TypeError

15 - Given the following code, what is calc.add(1, 2)?

```
class Calculator:
    def add(self, a, b, c=10):
        return a + b + c
calc = Calculator()
```

- ◯ Error due to missing third argument
- ◯ 3
- ✓ 13
- ◯ calc.add(1, 2, None)

16 - Which statement about inheritance is true?

- ◯ It forces all classes to have the same methods
- ✓ It helps avoid code duplication by reusing common behavior
- ◯ It removes the need for some methods
- ◯ It guarantees better performance

17 - Which statement about OOP is true?

- ◯ Replaces imperative paradigm
- ◯ Eliminates variables
- ◯ Removes the need for functions
- ✓ Adds an organizational layer on top of imperative programming

18 - Which UML notation represents composition?

- ◯ A plus sign before the class name
- ◯ A white diamond at the side of the container
- ✓ A black diamond at the side of the container
- ◯ A dashed arrow between methods

19 - Which statement best describes method overriding?

- ◯ A process in which the parent class method replaces the child class method at runtime
- ◯ A technique used to prevent method redefinition in subclasses to ensure consistency
- ◯ A way to specialize a class by adding new methods without altering existing ones
- ✓ A child class redefines a method from the parent class

20 - What is printed considering the following code?

```
class Example:
    def __init__(self):
        self.x = 10
    def modify(self):
        x = 20
        self.y = 30
e = Example()
e.modify()
print(e.x, e.y)
```

- ✓ 10 30
- ◯ 20 30

- ◯ 10 20
- ◯ AttributeError

21 - Which statement about method overloading in Python is correct?

- ◯ It is achieved by defining multiple methods with the same name and Python automatically selects the correct one
- ◯ It relies on runtime type checking to dispatch calls to different method implementations transparently
- ◯ It is implemented through inheritance, where subclasses redefine methods with different parameter lists
- ✓ It is simulated using default or variable arguments

22 - Direct access to an attribute is acceptable when:

- ◯ attribute is simple
- ◯ no validation is needed
- ◯ it is intended to be accessed directly as part of the class design
- ✓ all of these options

23 - Which situation most strongly suggests using methods instead of direct attribute access?

- ◯ Simple public data
- ✓ A rule must be enforced when assigning the value
- ◯ No validation needed

- ◯ Attribute never changes

24 - What happens when executing self.x = 10 inside __init__ for the first time?

- ✓ The attribute x is created for that instance and set to 10
- ◯ A local variable x is created
- ◯ The class attribute x is set to 10
- ◯ A global variable x is created

25 - Which is not a typical responsibility of a class?

- ◯ Representing a domain concept
- ✓ Controlling program execution flow
- ◯ Defining a template
- ◯ Defining behavior

26 - Given the following code, what should replace the blank so that it is printed the value 2?

```
class A:

    count = 0

    def __init__(self):

        _____
```

```
a1 = A()

a2 = A()

print(A.count)
```

- ◯ super().count += 1
- ◯ count = count+1
- ✓ A.count += 1
- ◯ self.count += 1

27 - Consider the following code, why is the relationship between Gym and Trainer considered aggregation?

```
class Trainer:

    def __init__(self, name):

        self.name = name

class Gym:

    def __init__(self, name):

        self.name = name

        self.trainers = []

    def add_trainer(self, trainer):
```

```
        self.trainers.append(trainer)
```

- ○ Trainer objects cannot exist without Gym
- ✓ Trainer objects can exist independently of Gym
- ○ Gym creates Trainer objects internally
- ○ Trainer inherits from Gym

28 - What is the key benefit of using classes instead of dictionaries and separate functions?

- ○ classes reduce the amount of code generally
- ○ classes prevent all logical errors
- ✓ classes group data and the operations that act on that data in a single structure
- ○ classes guarantee better runtime performance

29 - What is printed considering the following code?

```
class BankAccount:
    def __init__(self, balance):
        self._balance = balance
    def deposit(self, amount):
        if amount > 0:
```

```
        self._balance += amount

acc = BankAccount(100)

acc._balance = -500

acc.deposit(50)

print(acc._balance)
```

- ◯ -500
- ◯ 150
- ✓ -450
- ◯ 50

30 - Which statement correctly describes the relationship between a parent class and a child class?

- ◯ The child class replaces the parent class entirely
- ◯ The parent class depends on the child class
- ✓ The child class extends and specializes the parent class
- ◯ The parent class inherits from the child class

31 - Consider the following code, which type of relationship exists between Student and Course?

```
class Course:
```

```
    def __init__(self, name):
        self.name = name
class Student:
    def __init__(self, name):
        self.name = name
        self.courses = []
    def enroll(self, course):
        self.courses.append(course)
```

- ○ Composition
- ✓ Aggregation
- ○ Inheritance
- ○ Association

32 - Which type of relationship exists between Gym and Session?

```
class Session:
    def __init__(self, title):
        self.title = title
```

```
class Gym:
    def __init__(self, name):
        self.name = name
        self.sessions = []
    def create_session(self, title):
        session = Session(title)
        self.sessions.append(session)
```

- ○ Association
- ✓ Composition
- ○ Inheritance
- ○ Aggregation

33 - Which statement about Mermaid is correct?

- ○ It is only used for database diagrams
- ○ It requires a desktop installation and does not support documentation tools
- ✓ It can generate diagrams directly from code blocks and is easy to embed in documentation
- ○ It automatically converts Python code into executable programs

34 - Which statement about instances is correct?

- ◯ Instances cannot access methods
- ◯ All instances share attribute values
- ✓ Each instance has its own state
- ◯ Instances define class structure

35 - Given the following code, what happens if process is called directly?

```
class Payment:
    def process(self, amount):
        raise NotImplementedError()
```

- ◯ Returns None
- ✓ An exception is raised
- ◯ Returns 0
- ◯ Prints the amount

36 - Top-down design means:

- ◯ implementing low-level details first to test performance
- ✓ starting from a high-level description and refining it into smaller, more detailed parts

- ◯ focusing solely on optimizing the main algorithm
- ◯ avoiding decomposition to keep the code simple

37 - What is the purpose of raising NotImplementedError in a base class?

- ◯ To stop program execution permanently
- ◯ To prevent inheritance
- ◯ To define a default implementation
- ✓ To indicate that subclasses are expected to implement the method

38 - A class method operates on the ____ and receives ____ as the first parameter.

- ◯ instance / self
- ◯ class / self
- ✓ class / cls
- ◯ instance / cls

39 - Which statement about attributes and methods is correct?

- ◯ Methods store data, and attributes define the object's interface
- ✓ Attributes store behavior, and methods store the object's state
- ◯ Each class must be defined using both attributes and methods

- ◯ Attributes store the state of an object, and methods define its behavior

40 - Grouping values in tuples or lists improves organization, but this approach mainly suffers because:

- ◯ the data structure is not explicit
- ✓ all of these options
- ◯ the meaning of each position must be remembered or documented externally
- ◯ elements are accessed by position, which lacks semantic meaning and is error-prone

41 - When does a class become abstract in Python?

- ◯ When it defines methods that are overridden in subclasses
- ◯ When it raises NotImplementedError
- ✓ When it inherits from ABC and defines at least one @abstractmethod
- ◯ When it inherits from ABC

42 - What is an association between two classes?

- ✓ A structural relationship in which objects of one class hold references to objects of another class
- ◯ A method that automatically creates objects of both classes
- ◯ A mechanism to inherit all attributes and methods
- ◯ A rule that forces two classes to be in the same file

43 - Procedural programming organizes programs primarily around:

- ◯ control-flow statements
- ◯ modules and packages
- ✓ functions that operate on data
- ◯ variables and constants

44 - Consider the following code, which statement best describes the relationship between Student and Course?

```
class Course:
    def __init__(self, name):
        self.name = name
class Student:
    def __init__(self, name):
        self.name = name
        self.courses = []
    def enroll(self, course):
        self.courses.append(course)
```

- ◯ Course inherits from Student

- ◯ Course objects cannot exist without Student
- ✓ Student objects hold references to Course objects but do not control their lifecycle
- ◯ Student creates Course objects internally

45 - Why must methods include self as their first parameter?

- ◯ It is a keyword to keep the reference of the object from whom the method is called
- ◯ It is a keyword that refers to the class itself
- ✓ It is a reference to the current object instance, allowing the method to access its own attributes and other methods
- ◯ It allows the method to be executed without creating an object

46 - Given the following code, what is b.f()?

```
class A:
    def f(self):
        return 1
class B(A):
    pass
b = B()
```

- ✓ 1

- ◯ 0
- ◯ AttributeError
- ◯ None

47 - Which statement best describes aggregation?

- ✓ A weak whole-part relationship in which the contained objects can exist independently
- ◯ A strong whole-part relationship in which the container creates the parts
- ◯ A relationship that forbids object references
- ◯ A synonym for inheritance

48 - What is printed considering the following code?

```
class Counter:
    def __init__(self, start):
        self.value = start
    def increment(self):
        self.value += 1
c1 = Counter(10)
c2 = Counter(5)
```

```
c1.increment()

c1.increment()

c2.increment()

print(c1.value, c2.value)
```

- ○ 11 6
- ○ 11 5
- ✓ 12 6
- ○ 12 5

49 - Given the following code, what is g(B())?

```
class A:
    def f(self):
        return "A"
class B(A):
    def f(self):
        return "B"
def g(obj):
```

return obj.f()

- [x] B
- [ ] None
- [ ] Error
- [ ] A

50 - What happens if you try to instantiate a class that inherits from ABC and has an @abstractmethod?

- [ ] The object is created but calling the abstract method raises NotImplementedError
- [x] A TypeError is raised
- [ ] The class can be instantiated only if the abstract method is never called
- [ ] The abstract method is automatically given a default implementation

Enviar